Python Selenium

m2iformation.fr

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Python Selenium

Automatisation des Tests Web

Utopios® Tous droits réservés

Module 1

Fondamentaux Selenium & Installation

Objectifs: Installer Selenium, comprendre le WebDriver, écrire le premier script

Topics: Architecture, WebDriver, chromedriver, configuration, Hello World

Utopios® Tous droits réservés

Qu'est-ce que Selenium?

Une Suite d'Outils pour l'Automatisation Web

Selenium est un framework open-source qui automatise les navigateurs web. Idéal pour:

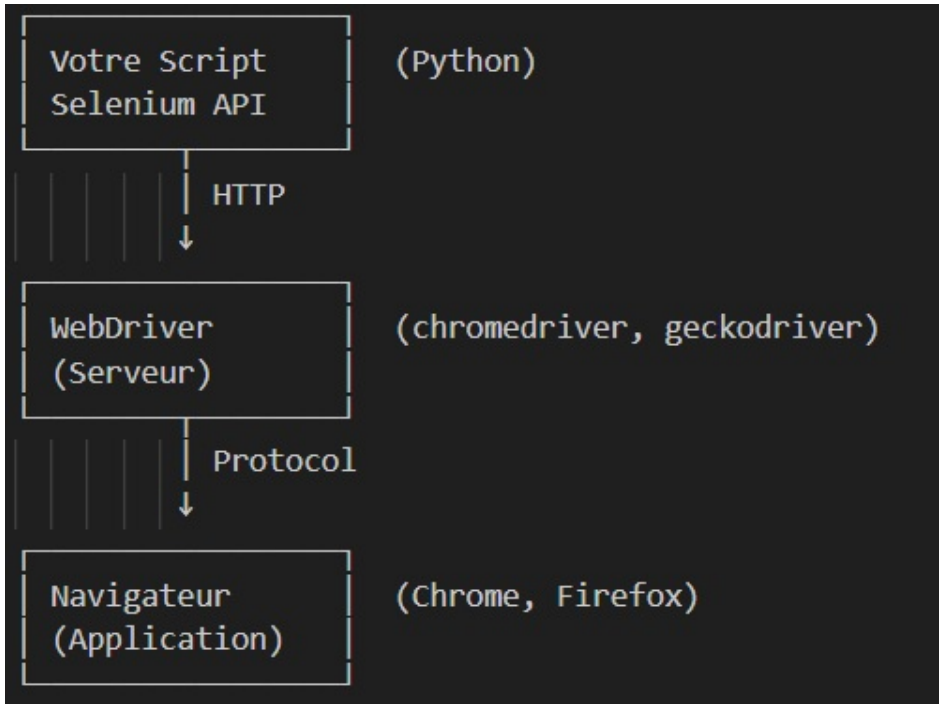
- **Tests automatisés:** Vérifier le fonctionnement d'applications web
- **Web scraping:** Extraire données de sites dynamiques
- **Automatisation RPA:** Automatiser des tâches web répétitives
- **Testing cross-browser:** Vérifier sur Chrome, Firefox, Safari, Edge

Avantage Python: Python + Selenium = code simple, lisible, rapide à développer. Parfait pour les testeurs!

Utopios® Tous droits réservés

Architecture Selenium

Comment ça marche?



Flux simple: Votre code Python → Requête WebDriver → Le WebDriver contrôle le navigateur

Utopios® Tous droits réservés

Installation Étape 1: Python

Vérifier et configurer Python

```
# Vérifier que Python 3.8+ est installé
python --version

# Créer un environnement virtuel
python -m venv venv

# Activer l'environnement
# macOS/Linux:
source venv/bin/activate

# Windows:
venv\Scripts\activate
```

Pourquoi un virtualenv? Isoler les dépendances du projet, éviter les conflits.

Utopios® Tous droits réservés

Installation Étape 2: Selenium

Installer les packages Python

```
# Installation basique
pip install selenium

# Avec outils de qualité (recommandé)
pip install selenium pytest webdriver-manager

# Vérifier l'installation
python -c "import selenium; print(selenium.__version__)"
```

⚠️ **Important:** Utilisez toujours un virtualenv pour les projets Python!

Utopios® Tous droits réservés

Installation Étape 3: WebDriver

Obtenir le driver du navigateur

Option A: WebDriver Manager (RECOMMANDÉ)

```
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.chrome.service import Service
from selenium import webdriver

# Télécharge automatiquement chromedriver
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service)
```

Utopios® Tous droits réservés

Installation Étape 3: WebDriver

Option B: Télécharger manuellement

- Chrome: https://chromedriver.chromium.org/
- Firefox: https://github.com/mozilla/geckodriver/releases
- Placer dans PATH ou spécifier le chemin

Utopios® Tous droits réservés

Premier Script

Hello World Selenium

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.chrome.service import Service

# 1. Ouvrir un navigateur
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service)

try:
    # 2. Aller sur un site
    driver.get("https://example.com")

    # 3. Vérifier le titre
    print(f"Titre: {driver.title}")

    # 4. Attendre un peu
    input("Appuyez sur Entrée pour fermer...")

finally:
    # 5. Fermer le navigateur
    driver.quit()
```

Utopios® Tous droits réservés

Exécuter le premier script

Pas à pas

1. Créer un fichier `hello_selenium.py`
2. Copier le code du slide précédent
3. Exécuter: `python hello_selenium.py`
4. Un navigateur Chrome s'ouvre automatiquement!
5. Appuyer sur Entrée pour fermer

Bravo! Vous avez exécuté votre premier script Selenium. Le navigateur s'est ouvert et a navigué vers example.com.

Utopios® Tous droits réservés

Concepts Clés

À retenir absolument

WebDriver

L'objet central qui contrôle le navigateur.

```
driver = webdriver.Chrome()
```

Éléments Web

Les objets HTML qu'on interagit. Texte, boutons, inputs, etc.

Sélecteurs

Comment trouver un élément: XPath, CSS, ID, class, etc.

Waits (Attentes)

Attendre que l'élément soit prêt avant agir. Crucial pour éviter les erreurs.

Utopios® Tous droits réservés

Checklist Installation

Vérifier que tout fonctionne

- ✅ Python 3.8+ installé
- ✅ Virtualenv créé et activé
- ✅ Selenium installé (`pip install selenium`)
- ✅ WebDriver Manager installé
- ✅ Chrome/Firefox à jour
- ✅ Premier script exécuté avec succès

Problème d'installation? Consultez la doc: https://www.selenium.dev/documentation/webdriver/getting_started/

Utopios® Tous droits réservés

Résumé Module 1

Qu'avez-vous appris?

- **Architecture Selenium:** Script → WebDriver → Navigateur
- **Installation:** Python virtualenv + Selenium + WebDriver
- **WebDriver Manager:** Gestion automatique des drivers
- **Premier script:** Ouvrir, naviguer, fermer un navigateur
- **Concepts clés:** WebDriver, éléments, sélecteurs, waits

Utopios® Tous droits réservés

Module 2

Sélecteurs XPath et CSS

Objectifs: Maîtriser XPath et CSS, trouver les éléments du DOM

Topics: Sélecteurs CSS, XPath simples/complexes, DevTools, stratégie de localisation

Utopios® Tous droits réservés

Localisation d'Éléments

Pourquoi est-ce important?

Avant d'interagir avec un élément, il faut le **trouver** dans la page.

Analogie:

Chercher un livre dans une bibliothèque. Vous devez connaître son emplacement précis (rayon, étagère, position).

```
# Sans sélecteur = Impossible de trouver l'élément
element = driver.find_element(...)  # Comment le trouver?

# Avec sélecteur = Localisé
element = driver.find_element(By.ID, "username")  # Trouvé!
element = driver.find_element(By.XPATH, "//button[@class='submit']")  # Trouvé!
```

Selenium supporte plusieurs stratégies: ID, NAME, CLASS_NAME, TAG_NAME, CSS_SELECTOR, XPATH

Utopios® Tous droits réservés

Stratégies de Localisation

Les 8 façons de trouver un élément

Stratégie	Syntaxe	Exemple
ID	By.ID	id="username"
NAME	By.NAME	name="password"
CLASS_NAME	By.CLASS_NAME	class="btn-primary"
TAG_NAME	By.TAG_NAME	`<button>`
CSS_SELECTOR	By.CSS_SELECTOR	`.form input[type="submit"]`
XPATH	By.XPATH	`//button[@class='submit']`
LINK_TEXT	By.LINK_TEXT	`<a>Cliquez ici</a>`
PARTIAL_LINK_TEXT	By.PARTIAL_LINK_TEXT	`<a>Cliquez</a>` (matching "Cliquez")

Utopios® Tous droits réservés

Sélecteurs Simples

ID, NAME, CLASS

HTML Exemple:

```
<input id="email" type="text" placeholder="Email">
<input name="password" type="password">
<button class="btn btn-primary" onclick="submit()">Login</button>
```

Localisation en Python:

```
from selenium.webdriver.common.by import By
# Par ID
email = driver.find_element(By.ID, "email")
# Par NAME
password = driver.find_element(By.NAME, "password")
# Par CLASS (première classe)
button = driver.find_element(By.CLASS_NAME, "btn-primary")
# Par TAG
buttons = driver.find_elements(By.TAG_NAME, "button")
```

✅ **Avantage:** Rapide et simple. Idéal quand l'élément a un ID ou NAME unique.

Utopios® Tous droits réservés

Sélecteurs CSS

Le choix du Web Developer

CSS Selectors sont la norme du web, flexibles et puissants.

```
# Sélecteur CSS basique
By.CSS_SELECTOR, "#email"              # Par ID
By.CSS_SELECTOR, ".btn-primary"        # Par classe
By.CSS_SELECTOR, "button"              # Par tag
# Sélecteurs composés
By.CSS_SELECTOR, "form button"         # Bouton dans form
By.CSS_SELECTOR, ".form > button"      # Enfant direct
By.CSS_SELECTOR, "input[type='email']" # Avec attribut
# Combinaisons
By.CSS_SELECTOR, ".container .btn-primary:hover"
By.CSS_SELECTOR, "button:nth-child(2)"  # 2ème bouton
By.CSS_SELECTOR, "input:not([disabled])" # Non-désactivé
```

💡 **Conseil:** CSS est plus léger que XPath et souvent plus rapide.

Utopios® Tous droits réservés

XPath Fondamentaux

Le langage de requête XML/HTML

XPath est plus puissant que CSS pour les recherches complexes.

```
# XPath absolu (rare, fragile)
/html/body/form/input[@name="email"]
# XPath relatif (recommandé)
//input[@name="email"]           # Chercher partout
//button[@class='submit']        # Par classe
//input[@type='text' and @required]  # Conditions multiples
# Chercher par texte
//button[text()='Login']
//a[contains(text(), 'Cliquez')]
# Chercher le parent/frère
//input[@name="email"]/..         # Parent
//button/following-sibling::span  # Frère suivant
```

⚠️ **Attention:** XPath est puissant mais peut être lent sur les gros DOM.

Utopios® Tous droits réservés

XPath Avancé

Patterns complexes

```
# Attributs partiels
//button[contains(@class, 'submit')]  # class contient "submit"
//input[starts-with(@name, 'user')]   # nom commence par "user"
//a[ends-with(@href, '.pdf')]         # lien finit par ".pdf"
# Positions et index
//button[1]                   # 1er bouton
//input[last()]               # Dernier input
//tr[position() > 5]          # Lignes après la 5ème
# Logique ET/OU
//button[@id='ok' and @type='submit']
//input[@required or @autofocus]
# Éléments imbriqués
//form[@id='login']//input[@type='password']
//div[@class='modal']//button[contains(@class, 'close')]
# Nombre d'enfants
//*[count(./button) > 3]       # Éléments avec +3 boutons
```

Utopios® Tous droits réservés

DevTools: Inspecter

Trouver les sélecteurs facilement

1. **Ouvrir DevTools:** F12 (Chrome/Firefox)
2. **Inspecter l'élément:** Clic droit → Inspecter
3. **Analyser le HTML:** Voir id, class, attributes
4. **Copier le sélecteur:** Clic droit sur l'élément → Copy selector

Utopios® Tous droits réservés

DevTools: Inspecter

Conseils DevTools:

- Utilisez la flèche d'inspection (coin haut-gauche)
- Vérifiez les IDs et classes dans l'inspect
- Testez le sélecteur dans la console

✅ **Test en console:** `document.querySelectorAll(".btn-primary")`

Utopios® Tous droits réservés

Choisir le Bon Sélecteur

Stratégie de localisation

Priorité

1. ID (unique, stable)
2. NAME (unique généralement)
3. CSS Selector (simple et rapide)
4. XPath (complexe mais puissant)

Utopios® Tous droits réservés

Choisir le Bon Sélecteur

⚠️ À éviter

- TAG_NAME seul (trop générique)
- XPath absolus (/html/body/...)
- Sélecteurs liés à l'UI (position, ordre)
- Éléments générés dynamiquement

Utopios® Tous droits réservés

Code Complet: Find Elements

Exemple réel avec gestion d'erreurs

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import NoSuchElementException, TimeoutException

# Trouver un seul élément
try:
    email_field = driver.find_element(By.ID, "email")
    print(f"Trouvé: {email_field}")
except NoSuchElementException:
    print("Email field not found!")

# Trouver tous les éléments
buttons = driver.find_elements(By.TAG_NAME, "button")
print(f"Nombre de boutons: {len(buttons)}")

# Attendre que l'élément soit présent
try:
    submit_btn = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.ID, "submit"))
    )
    submit_btn.click()
except TimeoutException:
    print("Submit button didn't appear within 10 seconds")
```

Utopios® Tous droits réservés

Résumé Module 2

Points clés

- **Localisation:** Trouver un élément par sélecteur
- **Stratégies:** ID, NAME, CLASS, CSS, XPATH, etc.
- **CSS Selectors:** Rapide, flexible, recommandé
- **XPath:** Puissant pour recherches complexes
- **DevTools:** Essentiel pour développer les sélecteurs
- **Stratégie:** ID/NAME > CSS > XPath

Utopios® Tous droits réservés

Module 3

Interactions Web Avancées

Objectifs: Maîtriser les actions utilisateur complexes

Topics: Clicks, drag-drop, formulaires, alerts, iframes, fenêtres

Utopios® Tous droits réservés

Actions Basiques

Click et Text

```
from selenium.webdriver.common.by import By
# Clic simple
button = driver.find_element(By.ID, "submit")
button.click()
# Remplir un champ de texte
email = driver.find_element(By.NAME, "email")
email.send_keys("test@example.com")
# Effacer un champ
email.clear()
email.send_keys("new_email@example.com")
# Soumettre un formulaire
form = driver.find_element(By.ID, "loginForm")
form.submit()
# Obtenir du contenu
text = element.text  # Texte visible
html = element.get_attribute("html")  # HTML
value = element.get_attribute("value")  # Valeur input
```

✅ **Cas d'usage:** Remplir les formulaires de login, recherche, etc.

Utopios® Tous droits réservés

ActionChains: Actions Composées

Interactions complexes

ActionChains permet les actions en chaîne: hover, drag, double-click, etc.

```
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.by import By
actions = ActionChains(driver)
# Double-click
element = driver.find_element(By.ID, "myElement")
actions.double_click(element).perform()
# Hover (survol souris)
menu = driver.find_element(By.CLASS_NAME, "dropdown-menu")
actions.move_to_element(menu).perform()
# Drag and drop
source = driver.find_element(By.ID, "draggable")
target = driver.find_element(By.ID, "droppable")
actions.drag_and_drop(source, target).perform()
# Combiné
actions.move_to_element(menu)
actions.click()
actions.send_keys("search term")
actions.perform()
```

Utopios® Tous droits réservés

Gérer les Fenêtres et Onglets

Windows et Windows Handles

```
# Obtenir la fenêtre courante
current_window = driver.current_window_handle
print(f"Fenêtre: {current_window}")
# Toutes les fenêtres
all_windows = driver.window_handles
print(f"Nombre de fenêtres: {len(all_windows)}")
# Ouvrir un nouvel onglet (lien avec target="_blank")
driver.execute_script("window.open('https://example.com')")
# Basculer vers une autre fenêtre
if len(all_windows) > 1:
    driver.switch_to.window(all_windows[1])
    print(f"Basculé vers onglet: {driver.title}")
# Retour à la première fenêtre
driver.switch_to.window(all_windows[0])
# Fermer l'onglet courant
driver.close()
# Fermer tout
driver.quit()
```

Utopios® Tous droits réservés

Gérer les Alerts

Alert, Confirm, Prompt

```
from selenium.webdriver.common.alert import Alert
# Alert simple (OK)
alert = Alert(driver)
alert.accept()  # Cliquer OK
# Alert avec texte
message = alert.text
print(f"Message: {message}")
# Confirm (OK/Cancel)
driver.find_element(By.ID, "deleteBtn").click()  # Déclenche confirm
alert = Alert(driver)
alert.dismiss()  # Cliquer Cancel
# ou alert.accept() pour OK
# Prompt (Texte + OK)
driver.find_element(By.ID, "promptBtn").click()
alert = Alert(driver)
alert.send_keys("Mon réponse")
alert.accept()
# Attendre une alert (avec wait)
from selenium.webdriver.support import expected_conditions as EC
WebDriverWait(driver, 10).until(EC.alert_is_present())
alert = driver.switch_to.alert
```

Utopios® Tous droits réservés

IFrames (Cadres Intégrés)

Naviguer dans les cadres

Les iframes sont des zones HTML isolées. Il faut les **"activer"** pour y interagir.

```
# HTML structure
# <iframe id="myFrame" src="..."></iframe>
# Basculer vers un iframe
frame = driver.find_element(By.ID, "myFrame")
driver.switch_to.frame(frame)
# Maintenant, interagir dans l'iframe
element = driver.find_element(By.ID, "elementInFrame")
element.click()
# Retour au contenu principal
driver.switch_to.default_content()
# Iframes imbriquées
driver.switch_to.frame("parentFrame")
driver.switch_to.frame("childFrame")  # Imbriqué
element = driver.find_element(By.ID, "deepElement")
# Retour
driver.switch_to.default_content     ()
```

⚠️ **Important:** Oublier de basculer vers l'iframe = NoSuchElementException!

Utopios® Tous droits réservés

Dropdown et Select

Sélectionner dans une liste

```
from selenium.webdriver.support.select import Select
# HTML: <select id="country"></select>
# Créer un objet Select
select = Select(driver.find_element(By.ID, "country"))
# Sélectionner par visible text
select.select_by_visible_text("France")
# Sélectionner par value (attribut)
select.select_by_value("FR")
# Sélectionner par index (0 = premier)
select.select_by_index(2)
# Obtenir l'option sélectionnée
selected = select.first_selected_option
print(f"Sélectionné: {selected.text}")
# Désélectionner (pour multi-select)
select.deselect_all()
select.deselect_by_visible_text("Option1")
# Toutes les options
all_options = select.options
for option in all_options:
    print(option.text)
```

Utopios® Tous droits réservés

Touches Clavier

Codes et combinaisons

```
from selenium.webdriver.common.keys import Keys
# Touches simples
element.send_keys(Keys.ENTER)       # Entrée
element.send_keys(Keys.TAB)         # Tabulation
element.send_keys(Keys.ESCAPE)      # Échapper
element.send_keys(Keys.SPACE)       # Espace
element.send_keys(Keys.DELETE)      # Supprimer
# Flèches
element.send_keys(Keys.ARROW_DOWN)
element.send_keys(Keys.ARROW_UP)
element.send_keys(Keys.ARROW_LEFT)
element.send_keys(Keys.ARROW_RIGHT)
# Ctrl+C, Ctrl+V, etc.
element.send_keys(Keys.CONTROL, 'a')  # Ctrl+A
element.send_keys(Keys.CONTROL, 'c')  # Ctrl+C
element.send_keys(Keys.SHIFT, Keys.TAB)  # Shift+Tab
# Combinaison sur ActionChains
actions = ActionChains(driver)
actions.key_down(Keys.CONTROL).send_keys('a').key_up(Keys.CONTROL).perform()
```

Utopios® Tous droits réservés

Charger des Fichiers

Upload de fichiers

```
# HTML: <input type="file" id="upload">
# Méthode 1: send_keys (chemin absolue)
file_input = driver.find_element(By.ID, "upload")
file_input.send_keys("/path/to/file.pdf")  # Chemin absolu
# Vérifier le fichier
uploaded_file = driver.find_element(By.ID, "uploadStatus")
assert "file.pdf" in uploaded_file.text
# Méthode 2: Avec multiple files
file_input.send_keys("/path/file1.pdf")
file_input.send_keys("/path/file2.pdf")
# Important: Le chemin DOIT être absolue
import os
path = os.path.abspath("my_document.txt")
file_input.send_keys(path)
```

💡 **Conseil:** Utilisez `os.path.abspath()` pour obtenir le chemin absolu.

Utopios® Tous droits réservés

Script JavaScript

Exécuter du JS depuis Python

```
# Exécuter du JavaScript
result = driver.execute_script("""
    return document.title;
""")
print(f"Titre: {result}")
# Avec arguments
element = driver.find_element(By.ID, "myElement")
driver.execute_script("arguments[0].scrollIntoView();", element)
# Retour de valeur
value = driver.execute_script("""
    return document.querySelector('#myInput').value;
""")
# Ajouter une classe CSS (pour testing)
driver.execute_script("""
    arguments[0].classList.add('highlight');
""", element)
# Attendre avec JavaScript
driver.execute_async_script("""
    var callback = arguments[arguments.length - 1];
    setTimeout(callback, 5000);  // Attendre 5 sec
""")
```

Utopios® Tous droits réservés

Exemple Réel: Formulaire Complet

Intégration de toutes les actions

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.select import Select
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.action_chains import ActionChains

driver = webdriver.Chrome()
driver.get("https://example.com/form")

# 1. Remplir le nom
name = driver.find_element(By.ID, "firstName")
name.send_keys("John Doe")

# 2. Sélectionner un pays
country_select = Select(driver.find_element(By.ID, "country"))
country_select.select_by_visible_text("France")

# 3. Cocher une checkbox
checkbox = driver.find_element(By.ID, "agree")
if not checkbox.is_selected():
    checkbox.click()

# 4. Charger un fichier
file_input = driver.find_element(By.NAME, "document")
file_input.send_keys("/tmp/document.pdf")

# 5. Cliquer le bouton
submit_btn = driver.find_element(By.XPATH, "//button[@type='submit']")
actions = ActionChains(driver)
actions.move_to_element(submit_btn).click().perform()

# 6. Gérer l'alert de confirmation
alert = Alert(driver)
alert.accept()

driver.quit()
```

Utopios® Tous droits réservés

Résumé Module 3

Points clés

- **Actions basiques:** click(), send_keys(), clear()
- **ActionChains:** double-click, hover, drag-drop
- **Fenêtres:** switch_to.window(), window_handles
- **Alerts:** accept(), dismiss(), send_keys()
- **IFrames:** switch_to.frame(), default_content()
- **Select:** select_by_visible_text(), select_by_value()
- **JavaScript:** execute_script() pour actions complexes

Utopios® Tous droits réservés

Module 4

Waits et Synchronisation

Objectifs: Gérer la synchronisation et les timeouts

Topics: Implicit waits, Explicit waits, Expected conditions, flaky tests

Utopios® Tous droits réservés

Synchronisation: Le Problème

Pourquoi attendre?

Les navigateurs web sont **asynchrones**. Les éléments peuvent apparaître/disparaître dynamiquement.

Code sans attente (MAUVAIS):

```
driver.get("https://example.com")
element = driver.find_element(By.ID, "loading-message")  # Erreur!
# L'élément n'existe pas encore...
```

Code avec attente (BON):

```
driver.get("https://example.com")
WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.ID, "loading-message"))
)
element = driver.find_element(By.ID, "loading-message")  # OK!
```

Utopios® Tous droits réservés

Types de Waits

3 approches

1. Implicit Wait (Basique)

Attendre globalement lors de `find_element()`. Moins flexible.

2. Explicit Wait (Recommandé)

Attendre spécifiquement sur une condition. Plus flexible et robuste.

3. Sleep (À ÉVITER)

Attendre un temps fixe. Lent et flaky.

Utopios® Tous droits réservés

Implicit Wait

Attendre par défaut

Une fois défini, TOUS les `find_element()` attendront ce temps.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
driver.implicitly_wait(10)  # 10 secondes par défaut
driver.get("https://example.com")
# find_element attendra 10 secondes l'élément
element = driver.find_element(By.ID, "submit")
element.click()
# Attention: S'applique à TOUS les find_element
# Peut rendre les tests plus lents
```

Limitation: Attend seulement la présence. Pas assez flexible pour des conditions complexes.

Utopios® Tous droits réservés

Explicit Wait (WebDriverWait)

Attendre une condition précise (RECOMMANDÉ)

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
# Attendre que l'élément soit présent dans le DOM
wait = WebDriverWait(driver, 10)  # 10 secondes max
element = wait.until(
    EC.presence_of_element_located((By.ID, "submit"))
)
element.click()
# Attendre que l'élément soit visible ET cliquable
button = wait.until(
    EC.element_to_be_clickable((By.XPATH, "//button[@class='action']"))
)
button.click()
# Attendre le titre
wait.until(EC.title_contains("Dashboard"))
# Attendre avec timeout court
wait = WebDriverWait(driver, 3)
element = wait.until(EC.presence_of_element_located((By.ID, "fast-element")))
```

Utopios® Tous droits réservés

Expected Conditions (EC)

Les conditions prédéfinies

```
from selenium.webdriver.support import expected_conditions as EC
# Présence (dans le DOM)
EC.presence_of_element_located((By.ID, "element"))
# Visibilité (affiché à l'écran)
EC.visibility_of_element_located((By.ID, "element"))
# Cliquabilité (visible ET activé)
EC.element_to_be_clickable((By.ID, "button"))
# Invisibilité (caché)
EC.invisibility_of_element_located((By.ID, "loading"))
# Texte dans un élément
EC.text_to_be_present_in_element((By.ID, "msg"), "Success")
# Titre de la page
EC.title_contains("Dashboard")
EC.title_is("Exact Title")
# Nombre d'éléments
EC.presence_of_all_elements_located((By.CLASS_NAME, "item"))
# URL de la page
EC.url_contains("example.com")
EC.url_matches(".*product.*")
```

Utopios® Tous droits réservés

Conditions Personnalisées

Créer vos propres conditions

Pour des vérifications spéciales non couverte par EC.

```python
from selenium.webdriver.support.ui import WebDriverWait

# Classe personnalisée
class WaitForPageLoad:
    def __call__(self, driver):
        return driver.execute_script(
            "return document.readyState"
        ) == "complete"

# Utilisation
wait = WebDriverWait(driver, 10)
wait.until(WaitForPageLoad())

# Ou une fonction lambda
wait.until(lambda d: d.find_element(By.ID, "data").text != "")

# Condition complexe: Element visible ET avec texte
class ElementVisibleWithText:
    def __init__(self, locator, text):
        self.locator = locator
        self.text = text

    def __call__(self, driver):
        element = driver.find_element(*self.locator)
        if element.is_displayed() and self.text in element.text:
            return element
        return False

wait.until(ElementVisibleWithText((By.ID, "status"), "Ready"))
```

Utopios® Tous droits réservés

Gestion d'Erreurs d'Attente

TimeoutException et NoSuchElementException

```python
from selenium.common.exceptions import TimeoutException, NoSuchElementException
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

wait = WebDriverWait(driver, 5)

try:
    # Si n'apparaît pas en 5 sec = TimeoutException
    element = wait.until(
        EC.presence_of_element_located((By.ID, "slow-element"))
    )
except TimeoutException:
    print("Element didn't appear within 5 seconds!")
    # Actions de secours
    driver.refresh()

try:
    # Si pas trouvé immédiatement = NoSuchElementException
    element = driver.find_element(By.ID, "missing")
except NoSuchElementException:
    print("Element doesn't exist on the page")

# Ou avec un défaut
try:
    element = wait.until(EC.presence_of_element_located((By.ID, "element")))
except TimeoutException:
    element = None

if element:
    element.click()
```

Utopios® Tous droits réservés

Patterns de Synchronisation

Cas réels communs

```
# Pattern 1: Attendre avant clic
button = wait.until(EC.element_to_be_clickable((By.ID, "submit")))
button.click()

# Pattern 2: Action puis attendre résultat
search_btn = driver.find_element(By.ID, "search")
search_btn.click()
results = wait.until(EC.presence_of_all_elements_located((By.CLASS_NAME, "result")))

# Pattern 3: Attendre disparition du loader
wait.until(EC.invisibility_of_element_located((By.CLASS_NAME, "spinner")))
# Maintenant la page est prête

# Pattern 4: Attendre changement d'URL
original_url = driver.current_url
driver.find_element(By.LINK_TEXT, "Next Page").click()
wait.until(EC.url_changes(original_url))

# Pattern 5: Attendre texte dynamique
wait.until(EC.text_to_be_present_in_element((By.ID, "status"), "Loaded"))
```

Utopios® Tous droits réservés

Sleep vs Wait

Pourquoi pas sleep()?

sleep() = MAUVAIS:

```
import time
time.sleep(10)  # Attend 10 sec TOUJOURS
element = driver.find_element(By.ID, "element")
# Si l'élément arrive en 2 sec, on attend quand même 10!
```

WebDriverWait = BON:

```
wait.until(EC.presence_of_element_located((By.ID, "element")))
# Cherche l'élément toutes les 500ms
# Si trouvé en 2 sec = continue immédiatement
# Si pas trouvé en 10 sec = TimeoutException
```

💡 **Résultat:** WebDriverWait est 4x plus rapide en moyenne!

Utopios® Tous droits réservés

Best Practices

Règles d'or

1. **Toujours use WebDriverWait, jamais sleep()**
2. **Utilisez element_to_be_clickable() avant click()**
3. **Utilisez presence_of_element_located() pour les assertions**
4. **Timeout court (3-5 sec) pour les tests rapides**
5. **Cibles stables:** ID > NAME > CSS > XPath
6. **Évitez les conditions complexes**

Résultat: Tests rapides, robustes, et fiables!

Utopios® Tous droits réservés

Résumé Module 4

Points clés

- **Synchronisation:** Attendre que le DOM soit prêt
- **Implicit Wait:** Basique, global
- **Explicit Wait (WebDriverWait):** Recommandé et flexible
- **Expected Conditions:** Conditions prédéfinies et personnalisées
- **Timeout Exceptions:** Gérer les erreurs
- **JAMAIS sleep():** Utiliser WebDriverWait

Utopios® Tous droits réservés

Module 5

Page Object Model (POM)

Objectifs: Organiser le code avec le pattern POM

Topics: Abstraction, Encapsulation, Maintenabilité, Réutilisabilité

Utopios® Tous droits réservés

Le Problème Sans POM

Code désordonné

MAUVAIS: Sélecteurs partout

```
# test_login.py
def test_login():
    driver.find_element(By.ID, "email").send_keys("test@example.com")
    driver.find_element(By.ID, "password").send_keys("pass123")
    driver.find_element(By.XPATH, "//button[@type='submit']").click()

# test_dashboard.py
def test_dashboard():
    driver.find_element(By.ID, "email").send_keys("test@example.com")
    driver.find_element(By.ID, "password").send_keys("pass123")
    driver.find_element(By.XPATH, "//button[@type='submit']").click()

# PROBLÈME: Code dupliqué, changement = partout!
```

Utopios® Tous droits réservés

Solution: Page Object Model

Encapsuler chaque page

BON: Sélecteurs centralisés

```
# pages/login_page.py
class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        self.email_input = (By.ID, "email")
        self.password_input = (By.ID, "password")
        self.submit_button = (By.XPATH, "//button[@type='submit']")

    def login(self, email, password):
        self.driver.find_element(*self.email_input).send_keys(email)
        self.driver.find_element(*self.password_input).send_keys(password)
        self.driver.find_element(*self.submit_button).click()

# test_login.py
def test_login():
    login_page = LoginPage(driver)
    login_page.login("test@example.com", "pass123")
```

Utopios® Tous droits réservés

Architecture POM

Structure recommandée

```
project/
├── pages/
│   ├── __init__.py
│   ├── base_page.py       # Classe mère
│   ├── login_page.py
│   ├── dashboard_page.py
│   └── product_page.py
├── tests/
│   ├── __init__.py
│   ├── test_login.py
│   ├── test_dashboard.py
│   └── test_checkout.py
├── conftest.py            # Fixtures pytest
└── requirements.txt
```

Utopios® Tous droits réservés

Architecture POM

Principes:

- Une classe par page/écran
- Sélecteurs comme attributs privés
- Méthodes publiques pour actions utilisateur

Utopios® Tous droits réservés

Base Page

Classe mère commune

```python
# pages/base_page.py
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class BasePage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)

    def find_element(self, locator):
        return self.driver.find_element(*locator)

    def find_elements(self, locator):
        return self.driver.find_elements(*locator)

    def wait_for_element(self, locator):
        return self.wait.until(
            EC.presence_of_element_located(locator)
        )

    def click(self, locator):
        self.wait.until(EC.element_to_be_clickable(locator)).click()

    def type_text(self, locator, text):
        element = self.wait_for_element(locator)
        element.clear()
        element.send_keys(text)

    def get_text(self, locator):
        return self.wait_for_element(locator).text
```

Utopios® Tous droits réservés

Login Page Implementation

Exemple concret

```python
# pages/login_page.py
from selenium.webdriver.common.by import By
from pages.base_page import BasePage

class LoginPage(BasePage):
    # Locators
    EMAIL_INPUT = (By.ID, "email")
    PASSWORD_INPUT = (By.ID, "password")
    SUBMIT_BUTTON = (By.XPATH, "//button[@type='submit']")
    ERROR_MESSAGE = (By.CLASS_NAME, "error-message")

    def __init__(self, driver):
        super().__init__(driver)
        self.driver.get("https://example.com/login")

    def login(self, email, password):
        """Effectuer un login"""
        self.type_text(self.EMAIL_INPUT, email)
        self.type_text(self.PASSWORD_INPUT, password)
        self.click(self.SUBMIT_BUTTON)

    def get_error_message(self):
        """Obtenir le message d'erreur"""
        return self.get_text(self.ERROR_MESSAGE)

    def is_error_displayed(self):
        """Vérifier si un erreur est affichée"""
        try:
            return self.wait_for_element(self.ERROR_MESSAGE).is_displayed()
        except:
            return False
```

Utopios® Tous droits réservés

Dashboard Page

Autre exemple

```python
# pages/dashboard_page.py
from selenium.webdriver.common.by import By
from pages.base_page import BasePage

class DashboardPage(BasePage):
    WELCOME_MESSAGE = (By.ID, "welcome")
    LOGOUT_BUTTON = (By.LINK_TEXT, "Logout")
    USER_STATS = (By.CLASS_NAME, "stat-card")

    def get_welcome_text(self):
        return self.get_text(self.WELCOME_MESSAGE)

    def logout(self):
        self.click(self.LOGOUT_BUTTON)

    def get_all_stats(self):
        elements = self.wait.until(
            EC.presence_of_all_elements_located(self.USER_STATS)
        )
        return [elem.text for elem in elements]

    def is_logged_in(self):
        try:
            self.wait_for_element(self.WELCOME_MESSAGE)
            return True
        except:
            return False
```

Utopios® Tous droits réservés

Tests avec POM

Code test propre

```python
# tests/test_login.py
import pytest
from pages.login_page import LoginPage
from pages.dashboard_page import DashboardPage

@pytest.fixture
def driver():
    # Setup
    driver = webdriver.Chrome()
    yield driver
    # Teardown
    driver.quit()

def test_valid_login(driver):
    login_page = LoginPage(driver)
    login_page.login("test@example.com", "password123")

    dashboard = DashboardPage(driver)
    assert dashboard.is_logged_in()
    assert "Welcome" in dashboard.get_welcome_text()

def test_invalid_credentials(driver):
    login_page = LoginPage(driver)
    login_page.login("invalid@example.com", "wrongpass")

    assert login_page.is_error_displayed()
    assert "Invalid credentials" in login_page.get_error_message()
```

Utopios® Tous droits réservés

Avantages du POM

Pourquoi c'est important

- **Maintenabilité:** Changer sélecteur = une seule place
- **Réutilisabilité:** Réutiliser pages dans plusieurs tests
- **Lisibilité:** Tests lisent comme du français
- **Abstraction:** Tests ne connaissent pas les détails DOM
- **Scalabilité:** Facile d'ajouter nouvelles pages
- **Stabilité:** Waits/erreurs gérées au niveau page

Utopios® Tous droits réservés

Advanced: Page Hierarchy

Pages complexes

```python
# pages/product_page.py
from pages.base_page import BasePage

class ProductPage(BasePage):
    PRODUCT_TITLE = (By.CLASS_NAME, "product-title")
    ADD_TO_CART_BUTTON = (By.XPATH, "//button[text()='Add to Cart']")
    QUANTITY_INPUT = (By.ID, "quantity")
    PRICE = (By.CLASS_NAME, "product-price")

    def add_to_cart(self, quantity=1):
        if quantity > 1:
            self.type_text(self.QUANTITY_INPUT, str(quantity))
        self.click(self.ADD_TO_CART_BUTTON)

    def get_product_info(self):
        return {
            'title': self.get_text(self.PRODUCT_TITLE),
            'price': self.get_text(self.PRICE)
        }

    def get_total_price(self, quantity):
        price_text = self.get_text(self.PRICE)
        price = float(price_text.replace('$', ''))
        return price * quantity
```

Utopios® Tous droits réservés

Best Practices POM

Règles d'or

1. **Une classe par page/composant logique**
2. **Sélecteurs CONSTANTS en MAJUSCULES**
3. **Méthodes = actions utilisateur, pas assertions**
4. **BasePage pour code commun**
5. **Exceptions en init si page spécifique**
6. **Retourner la prochaine page après action**

Résultat: Code maintenable et scalable!

Utopios® Tous droits réservés

Résumé Module 5

Points clés

- **Page Object Model:** Design pattern pour tests
- **Encapsulation:** Sélecteurs dans les classes
- **BasePage:** Code commun à toutes les pages
- **Abstraction:** Tests lisent comme du métier
- **Maintenabilité:** Changements centralisés
- **Scalabilité:** Facile d'ajouter des tests

Utopios® Tous droits réservés

Module 6

Tests & Assertions Selenium

Objectifs: Valider et tester vos automations

Topics: Assertions, validations, vérifications, rapports

Utopios® Tous droits réservés

Pourquoi Tester?

Validation et Assurance Qualité

Les tests vérifient que votre automation fonctionne comme prévu.

Avantages:

- **Confiance:** Vérifier que le site fonctionne
- **Automatisation:** Exécuter les vérifications automatiquement
- **Maintenance:** Détecter les régressions
- **Documentation:** Les tests documentent le comportement attendu

Utopios® Tous droits réservés

Assertions Basiques

Vérifier les conditions en Selenium

```
# Vérifier le titre
assert driver.title == "Google"

# Vérifier l'URL
assert "google.com" in driver.current_url

# Vérifier un élément existe
search_box = driver.find_element(By.NAME, "q")
assert search_box is not None

# Vérifier le texte
welcome = driver.find_element(By.ID, "welcome")
assert "Welcome John" in welcome.text

# Vérifier plusieurs éléments
results = driver.find_elements(By.CSS_SELECTOR, "div.g")
assert len(results) > 0

# Vérifier un attribut
submit = driver.find_element(By.ID, "submit")
assert submit.get_attribute("disabled") is None

# Vérifier la visibilité
element = driver.find_element(By.ID, "content")
assert element.is_displayed()
```

Utopios® Tous droits réservés

Vérifications Courantes

Les cas d'usage les plus fréquents

```python
# 1. Présence d'un élément
try:
    element = driver.find_element(By.ID, "alert")
    assert element is not None
    print("✅ Élément trouvé")
except:
    print("❌ Élément non trouvé")

# 2. Absence d'un élément
elements = driver.find_elements(By.CLASS_NAME, "error")
assert len(elements) == 0, "Erreurs présentes"

# 3. Texte attendu
title = driver.find_element(By.TAG_NAME, "h1")
assert title.text == "Welcome to Google"

# 4. Nombre d'éléments
items = driver.find_elements(By.CLASS_NAME, "item")
assert len(items) == 10, f"Expected 10, got {len(items)}"

# 5. État disabled/enabled
button = driver.find_element(By.ID, "submit")
assert button.is_enabled()

# 6. Classe CSS
div = driver.find_element(By.ID, "content")
assert "active" in div.get_attribute("class")
```

Utopios® Tous droits réservés

Gestion des Erreurs

Tester et capturer les erreurs

```
# Try/Except pour assertions
try:
    driver.get("https://example.com")
    title = driver.find_element(By.TAG_NAME, "h1")
    assert "Home" in title.text
    print("✅ Test passé")

except AssertionError as e:
    print(f"❌ Assertion échouée: {e}")

except Exception as e:
    print(f"❌ Erreur: {e}")
    import traceback
    traceback.print_exc()

finally:
    print("Fermeture...")
    driver.quit()
```

💡 **Toujours:** Utiliser try/except/finally pour nettoyer

Utopios® Tous droits réservés

Fonction de Test

Organiser les vérifications

```python
def test_google_search():
    """Test la recherche sur Google"""
    driver = webdriver.Chrome()

    try:
        # ARRANGE: Setup
        driver.get("https://google.com")

        # ACT: Agir
        search_box = driver.find_element(By.NAME, "q")
        search_box.send_keys("Selenium Python")
        search_box.send_keys(Keys.RETURN)
        time.sleep(2)

        # ASSERT: Vérifier
        results = driver.find_elements(By.CSS_SELECTOR, "div.g")
        assert len(results) > 0, "Aucun résultat"
        print("✅ Test réussi")
        return True

    except Exception as e:
        print(f"❌ Test échoué: {e}")
        return False

    finally:
        driver.quit()

# Exécuter
if __name__ == "__main__":
    success = test_google_search()
    exit(0 if success else 1)
```

Utopios® Tous droits réservés

Assertions Utiles

Snippets pour valider

```
# Pages
assert driver.title != ""
assert "login" in driver.current_url

# Éléments
elem = driver.find_element(By.ID, "btn")
assert elem is not None
assert elem.is_displayed()
assert elem.is_enabled()

# Texte
assert "Welcome" in driver.page_source
assert elem.text.strip() != ""

# Attributs
assert elem.get_attribute("href") == "/"
assert elem.get_attribute("class") == "active"
assert elem.get_attribute("data-id") == "123"

# Collections
items = driver.find_elements(By.CLASS_NAME, "item")
assert len(items) >= 5
assert items[0].text == "First"

# Comparaisons
price = float(elem.text.replace("$", ""))
assert price > 0
assert price < 1000
```

Utopios® Tous droits réservés

Rapports de Test

Documenter les résultats

```python
import json
from datetime import datetime

def run_tests_with_report():
    """Exécuter les tests et générer un rapport"""
    report = {
        "timestamp": datetime.now().isoformat(),
        "tests": [],
        "passed": 0,
        "failed": 0
    }

    test_cases = [
        ("google_search", test_google_search),
        ("wikipedia_navigation", test_wiki_nav),
        ("github_login", test_github_login),
    ]

    for test_name, test_func in test_cases:
        try:
            test_func()
            report["tests"].append({"name": test_name, "status": "PASS"})
            report["passed"] += 1
            print(f"✅ {test_name}")
        except Exception as e:
            report["tests"].append({"name": test_name, "status": "FAIL", "error": str(e)})
            report["failed"] += 1
            print(f"❌ {test_name}: {e}")

    # Sauvegarder le rapport
    with open("test_report.json", "w") as f:
        json.dump(report, f, indent=2)

    print(f"\n📊 Résultats: {report['passed']} passé, {report['failed']} échoué")

run_tests_with_report()
```

Utopios® Tous droits réservés

Best Practices

Tester correctement

1. **Un test = un cas:** Tester une fonctionnalité
2. **Noms clairs:** test_search_returns_results()
3. **Arrange-Act-Assert:** Setup → Action → Vérification
4. **Messages explicites:** assert x, "Pourquoi ça échoue"
5. **Gestion d'erreurs:** Try/except/finally
6. **Nettoyage:** Toujours fermer le driver
7. **Attendre:** WebDriverWait avant assertions

Utopios® Tous droits réservés

Résumé Module 6

Points clés

- **Assertions:** Vérifier les conditions
- **Try/Except:** Gérer les erreurs
- **Arrange-Act-Assert:** Structure de test
- **Vérifications courantes:** Titre, URL, éléments, texte
- **Rapports:** Documenter les résultats
- **Best Practices:** Noms clairs, messages explicites

Utopios® Tous droits réservés

Module 7

Gestion d'Erreurs et Screenshots

Objectifs: Déboguer et documenter les tests

Topics: Try-except, Logging, Screenshots, Vidéos, Rapports

Utopios® Tous droits réservés

Exceptions Courantes

Erreurs à gérer

```
from selenium.common.exceptions import (
    NoSuchElementException,       # Élément pas trouvé
    TimeoutException,              # Timeout du wait
    StaleElementReferenceException,# Élément DOM invalidé
    ElementNotVisibleException,    # Élément pas visible
    InvalidSelectorException       # Sélecteur invalide
)

try:
    element = driver.find_element(By.ID, "missing")
except NoSuchElementException:
    print("Element not found")

try:
    wait.until(EC.presence_of_element_located((By.ID, "slow")))
except TimeoutException:
    print("Element took too long to appear")

try:
    element.click()
except StaleElementReferenceException:
    print("Element is stale, refind it")
```

Utopios® Tous droits réservés

Pattern Try-Except

Gestion d'erreurs robuste

```python
from selenium.common.exceptions import NoSuchElementException

def safe_find_element(driver, locator):
    """Chercher un élément, retourner None s'il manque"""
    try:
        return driver.find_element(*locator)
    except NoSuchElementException:
        return None

def safe_get_text(driver, locator, default=""):
    """Obtenir le texte, retourner default s'il manque"""
    try:
        element = driver.find_element(*locator)
        return element.text
    except (NoSuchElementException, AttributeError):
        return default

# Utilisation
email = safe_find_element(driver, (By.ID, "email"))
if email:
    email.send_keys("test@example.com")
else:
    print("Email field not found")

message = safe_get_text(driver, (By.ID, "message"), "No message")
```

Utopios® Tous droits réservés

Screenshots

Capturer l'écran pour debugging

```
import os
from datetime import datetime

# Screenshot simple
driver.save_screenshot("screenshot.png")

# Avec timestamp
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
driver.save_screenshot(f"screenshot_{timestamp}.png")

# Créer un dossier screenshots
os.makedirs("screenshots", exist_ok=True)
driver.save_screenshot(f"screenshots/test_login_{timestamp}.png")

# Screenshot d'un élément spécifique
element = driver.find_element(By.ID, "myElement")
element.screenshot(f"element_{timestamp}.png")

# Lors d'une erreur
def take_error_screenshot(driver, test_name):
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    path = f"screenshots/error_{test_name}_{timestamp}.png"
    driver.save_screenshot(path)
    print(f"Error screenshot: {path}")
```

Utopios® Tous droits réservés

Logging

Tracer l'exécution

```python
import logging

# Configurer logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("tests.log"),
        logging.StreamHandler()  # Console aussi
    ]
)

logger = logging.getLogger(__name__)

# Utilisation
def test_login(driver):
    logger.info("Starting login test")

    login_page = LoginPage(driver)
    logger.info("LoginPage initialized")

    try:
        login_page.login("user@example.com", "pass123")
        logger.info("Login successful")
    except Exception as e:
        logger.error(f"Login failed: {e}")
        raise

logger.warning("This is slow, monitor performance")
logger.debug("Detailed debug info for developers")
```

Utopios® Tous droits réservés

Fixture avec Screenshot

Capturer automatiquement en cas d'erreur

```python
# conftest.py
import pytest
import os
from datetime import datetime

@pytest.fixture
def driver(request):
    """Driver avec screenshot auto en cas d'erreur"""
    driver = webdriver.Chrome()
    yield driver

    # Teardown: Si le test a échoué, prendre screenshot
    if request.node.rep_call.failed:
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        test_name = request.node.name
        path = f"screenshots/error_{test_name}_{timestamp}.png"

        os.makedirs("screenshots", exist_ok=True)
        driver.save_screenshot(path)
        print(f"\n❌ Screenshot: {path}")

    driver.quit()

# Ou avec pytest hook
@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    rep = outcome.get_result()
    if rep.failed and hasattr(item, "funcargs"):
        driver = item.funcargs.get("driver")
        if driver:
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            driver.save_screenshot(f"screenshots/error_{item.name}_{timestamp}.png")
```

Utopios® Tous droits réservés

Enregistrer Vidéos

Enregistrer les tests (avancé)

```
# Pour enregistrer avec pytest-video
# Installation: pip install pytest-video

# conftest.py
import pytest
from pyvirtualdisplay import Display
import pyvnc

@pytest.fixture(scope="function")
def driver_with_video(request):
    """Driver avec enregistrement vidéo"""
    display = Display(visible=0, size=(1920, 1080))
    display.start()

    driver = webdriver.Chrome()

    # Démarrer enregistrement
    from pytest_video import Record
    recorder = Record(f"videos/{request.node.name}.mp4")
    recorder.start()

    yield driver

    # Arrêter et sauvegarder
    recorder.stop()
    display.stop()
    driver.quit()

# Alternative: Utiliser allure pour les vidéos
# pip install allure-pytest
# pytest --alluredir=allure-results
```

Utopios® Tous droits réservés

Rapports Pytest

Générer des rapports

```
# Rapport HTML
pytest --html=report.html --self-contained-html tests/

# Rapport Allure (plus joli)
pip install allure-pytest
pytest --alluredir=allure-results tests/
allure serve allure-results

# Rapport JUnit (pour CI/CD)
pytest --junit-xml=results.xml tests/

# Options combinées
pytest \
    -v \
    --html=report.html \
    --self-contained-html \
    --junit-xml=results.xml \
    --alluredir=allure-results \
    tests/

# Voir le rapport
open report.html  # macOS
start report.html  # Windows
```

Utopios® Tous droits réservés

Assertions Améliorées

Messages clairs en cas d'échec

```python
# Message explicite
assert user.is_active, f"User {user.name} should be active"

# Multiple assertions avec pré-conditions
def test_user_creation(driver):
    user = create_user(driver, "John", "john@example.com")

    # Assertions avec contexte
    assert user.id is not None, "User ID should be created"
    assert user.name == "John", f"Expected 'John', got '{user.name}'"
    assert user.email == "john@example.com", \
        f"Email mismatch: expected john@example.com, got {user.email}"
    assert user.is_active, "User should be active by default"

# Avec pytest-assert-msg
import pytest
assert condition, "This will be more helpful"

# Assertions dans une fixture
@pytest.fixture
def verify_login(driver):
    dashboard = DashboardPage(driver)

    assert dashboard.is_visible(), "Dashboard should be visible after login"
    assert dashboard.get_username() != "", "Username should be displayed"

    return dashboard
```

Utopios® Tous droits réservés

Retry Logic

Réessayer les tests flaky

```
import pytest
from tenacity import retry, stop_after_attempt, wait_fixed

# Avec décorateur tenacity
@retry(stop=stop_after_attempt(3), wait=wait_fixed(2))
def flaky_operation(driver):
    """Réessayer 3 fois, attendre 2 sec entre tentatives"""
    element = driver.find_element(By.ID, "sometimes-appears")
    element.click()

# Ou avec pytest-rerunfailures
# pip install pytest-rerunfailures

@pytest.mark.flaky(reruns=3)  # Réessayer 3 fois
def test_flaky_test(driver):
    # Test qui peut échouer occasionnellement
    result = unstable_operation(driver)
    assert result == "success"

# Exécution
# pytest --reruns 3 --reruns-delay 2 test_flaky.py
```

Utopios® Tous droits réservés

Checklist Debugging

Quand un test échoue

1. **Vérifier le screenshot:** Y a-t-il une erreur visuelle?
2. **Lire les logs:** Quel était l'état du système?
3. **Rejouer le test:** Est-ce reproductible?
4. **Mode headless off:** Voir le navigateur en action
5. **Ajouter du logging:** `logger.debug()` partout
6. **Augmenter timeouts:** Peut-être trop rapide?
7. **Vérifier les sélecteurs:** DevTools en direct

Utopios® Tous droits réservés

Résumé Module 7

Points clés

- **Exceptions:** Gérer `NoSuchElementException`, `TimeoutException`
- **Try-Except:** Code robuste avec fallbacks
- **Screenshots:** Capturer l'écran en cas d'erreur
- **Logging:** Tracer l'exécution des tests
- **Rapports:** HTML, Allure, JUnit pour CI/CD
- **Assertions claires:** Messages d'erreur explicites
- **Debugging:** Screenshots, logs, rejeu manuel

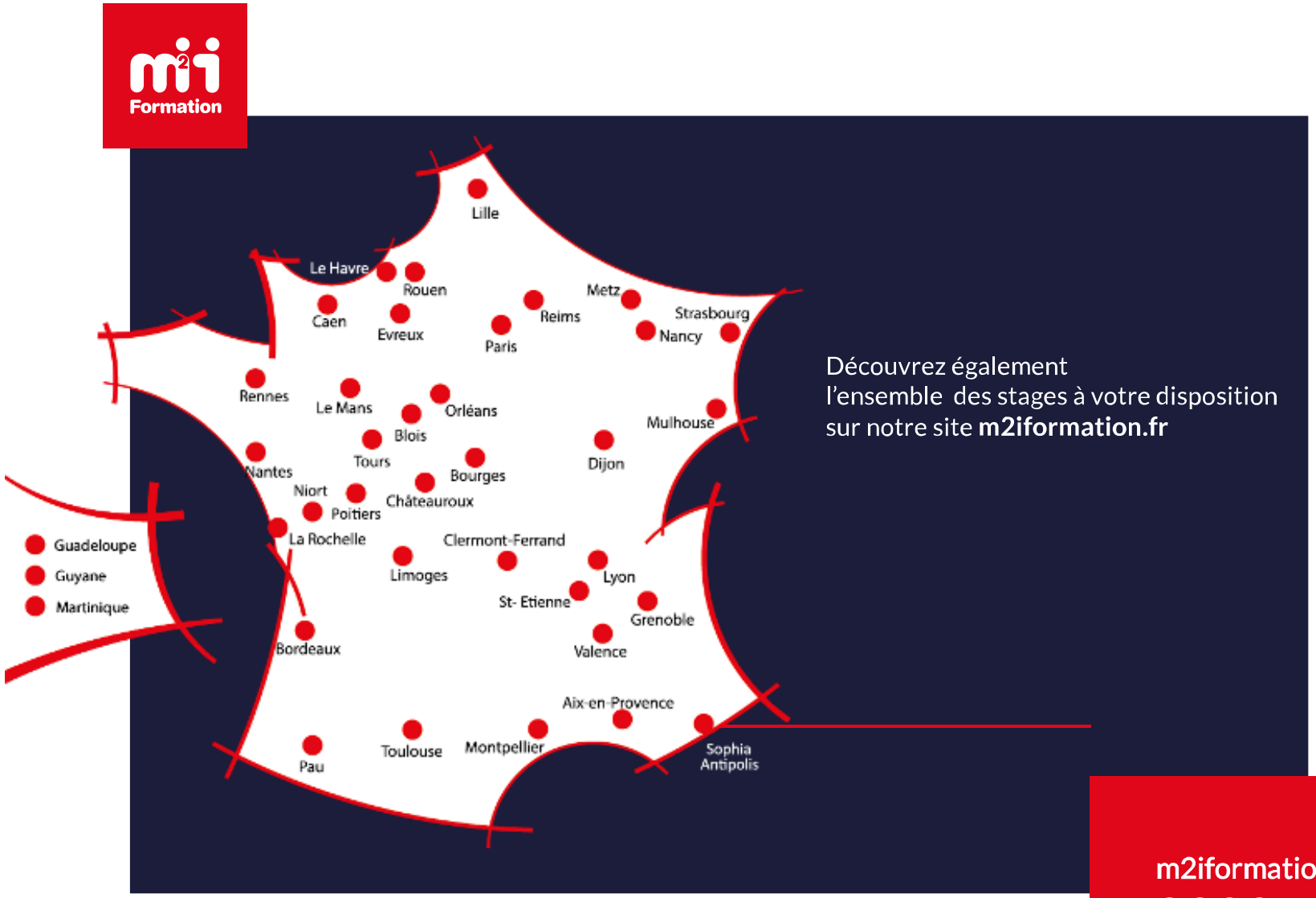
Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Merci pour votre attention

Des questions ?

Utopios® Tous droits réservés



Découvrez également
l'ensemble des stages à votre disposition
sur notre site **m2information.fr**

m2iformation.fr